OnlineBankingApp web front-end design

Oct 25, 2026 · @Kevin

Overview

OnlineBankingApp Web is a Next.js App Router application in React 19 and TypeScript. It covers the four requirement areas: sign in, accounts overview, transaction history and transfers. It needs **one small change to the OnlineBankingAPI**, its session cookie path (now `/`, see Sign-in), and nothing else: the browser only ever talks to the Next.js origin, and Next.js forwards `/api/*` to the ASP.NET Core API, so the API's existing session cookie and Problem Details errors work as intended without the need for a CORS policy.

- **State.** One Redux store, built with Redux Toolkit. RTK Query holds all API data. Two small slices hold client state: the signed-in customer, and the in-progress transfer.
- **Rendering.** Pages are server-rendered shells. Account and transfer data is fetched in the browser through RTK Query. The session cookie is `HttpOnly` with `Path=/`, so it also reaches the Next.js server on page requests, but there it is used only for the `proxy.ts` pre-check (see Sign-in).
- **Build.** Next.js 16 builds with its default bundler, Turbopack, for both `dev` and `build`. No bundler configuration is needed.
- **Contract.** RTK Query endpoints and hooks are generated from the API's own OpenAPI document (`/openapi/v1.json`), so a contract change shows up as a compile error.
- **Exactly-once transfers.** The UI follows the API's quote, confirm and execute flow and its `Idempotency-Key` client contract, so a double click, retry or refresh never moves money twice.
- **Scope.** A demo prototype for the seeded customers (alice, ben, chloe), on desktop and phone widths.

Recommended toolchain

Every tool below works with React 19 and Next.js 16. Versions are major lines; pin the latest patch of each when the project is created.

Area	Tool	Version line	Why
Runtime	Node.js	24 LTS	Current LTS, required by Next.js 16
Package manager	npm	11	Ships with Node.js 24, package-lock.json in repo (npm 10 cannot install the dependency tree)
Framework	Next.js (App Router)	16	React 19 support, `rewrites` for the API proxy, file-based routes
Bundler	Turbopack (built into Next.js)	ships with Next.js 16	Default for `next dev` and `next build`. Fast incremental dev rebuilds, no configuration needed.
UI library	React, React DOM	19.2	Required by Next.js 16
Language	TypeScript	5.9+ (`strict`)	Type safety end to end with the generated API endpoints
Styling	Tailwind CSS	4	Utility CSS with no runtime, built through PostCSS, which Turbopack supports out of the box; the default for shadcn/ui
Components	shadcn/ui on Radix UI	current CLI	Accessible dialog, select, date picker, table and form parts copied into the repo, so they can be restyled freely
Icons	lucide-react	current	Icon set used by shadcn/ui
Toasts	sonner	2	Transfer result and error toasts, ships with shadcn/ui

Area	Tool	Version line	Why
State	Redux Toolkit + react-redux	2 / 9	One store: `configureStore`, slices, listener middleware for persistence, Redux DevTools
Server data	RTK Query (in Redux Toolkit)	2	Caching, tag-based invalidation after a transfer, and infinite-query endpoints for cursor paging
API endpoints	@rtk-query/codegen-openapi	2	Generates typed RTK Query endpoints and hooks from `/openapi/v1.json`
Forms	React Hook Form + @hookform/resolvers	7	Sign-in and transfer forms without re-render churn
Validation	Zod	4	Client-side mirror of the API's transfer rules, shared by forms and tests
URL state	nuqs	2	History filters (`from`, `to`, `type`, `q`) live in the URL, so they survive refresh and can be shared
Dates	date-fns + @date-fns/tz	4	Date-range presets and display in the bank's time zone (`America/New_York`)
Unit and component tests	Vitest + React Testing Library	4 / 16	Fast tests against a real store in jsdom (Vitest uses its own Vite pipeline, independent of the app build)
API mocking	MSW (Mock Service Worker)	2	The same mocks drive component tests and an offline demo mode

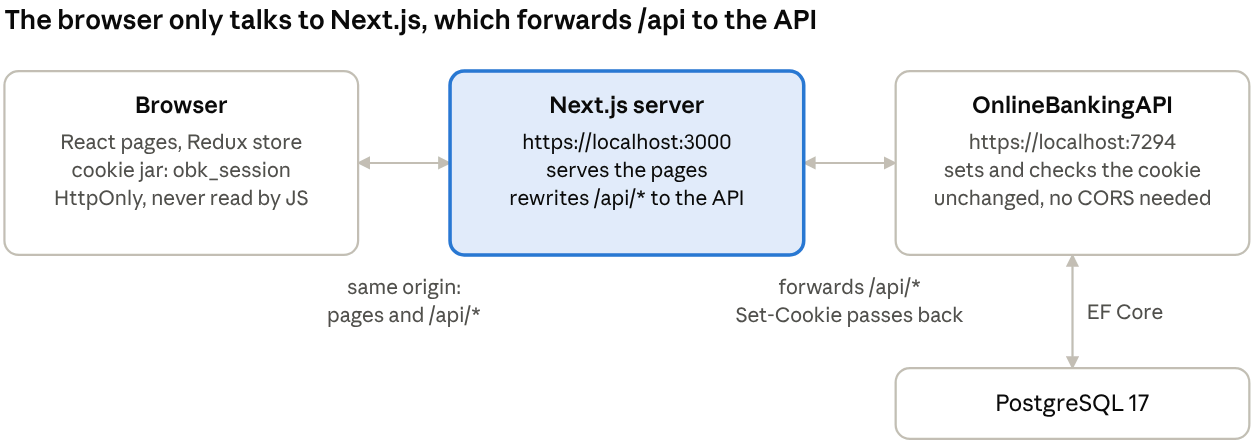
Area	Tool	Version line	Why
End-to-end tests	Playwright + @axe-core/playwright	1.x	Real browser against the real API and seeded data, plus accessibility scans
Lint	ESLint with `eslint-config-next`	9 (flat config)	Next.js, React Hooks and jsx-a11y rules
Format	Prettier + prettier-plugin-tailwindcss	3	Consistent formatting and sorted class names

Left out on purpose:

- **TanStack Query.** RTK Query covers the same ground, and two server caches would disagree.
- **Redux Persist.** Only the transfer draft is persisted, through a short listener.
- **Axios.** RTK Query's `fetchBaseQuery` is enough.
- **NextAuth/Auth.js.** The API already owns sessions.
- **Webpack.** Nothing in this design needs a Webpack plugin or loader, and Turbopack's dev loop is much faster. Adding `--webpack` to the scripts would switch back if that ever changes.

Integration with OnlineBankingAPI

Next.js acts as a same-origin proxy: a `rewrites` rule forwards every `/api/*` request to the API. To the browser, the API lives on the web app's own origin. The API does not need a CORS policy and sets `obk_session` with `SameSite=Strict; Path=/`, so a same-origin proxy is the setup that works with it unchanged.



runtime architecture · browser, Next.js proxy, API, database

Only the Next.js server knows where the API is. The browser sees a single origin, so the strict cookie and the absence of a CORS policy are by-design.

```
// next.config.ts
import type { NextConfig } from "next";

const apiOrigin = process.env.API_ORIGIN ?? "https://localhost:7294";

const config: NextConfig = {
  async rewrites() {
    return [{ source: "/api/:path*", destination: `${apiOrigin}/api/:path*`
}];
  },
};

export default config;
```

Session cookie

- `POST /api/v1/auth/login` returns `Set-Cookie: obk_session=…; HttpOnly; Secure; SameSite=Strict; Path=/`. The proxy passes it through, and the browser stores it for the web origin.
- Every later `fetch("/api/…")` sends the cookie automatically. JavaScript never sees the token, and the app ignores the `sessionToken` in the login body.
- Because the cookie path is `/`, page requests such as `/accounts` carry it too. `proxy.ts` uses that for a fast pre-check before any page renders (see Sign-in), and the API stays the real gate.

HTTPS in development

- Run the web app with `next dev --experimental-https` (https://localhost:3000), so the `Secure` cookie is accepted in every browser, Safari included.
- The proxy calls the API's https profile (https://localhost:7294). Node must trust the ASP.NET Core development certificate: run `dotnet dev-certs https --export-path ./certs/aspnet-dev.pem --format Pem --no-password` once and set `NODE_EXTRA_CA_CERTS=./certs/aspnet-dev.pem` in `.env.development.local`.
- Do not point the proxy at the http profile: the API's `UseHttpsRedirection` answers with a redirect.

Redux store and typed API

- **Generated endpoints.** `npm run api:codegen` runs `@rtk-query/codegen-openapi` against `https://localhost:7294/openapi/v1.json`. It writes `src/api/bankApi.generated.ts`: typed endpoints and hooks such as `useGetAccountsQuery`, `useCreateTransferQuoteMutation` and `useExecuteTransferMutation`. The generated file is committed, and CI fails if regenerating it changes anything.
- **Base API.** `src/api/baseApi.ts` holds one `createApi` with `fetchBaseQuery({ baseUrl: "" })` (same origin, so the cookie is sent), wrapped in `bankBaseQuery`. The wrapper turns `application/problem+json` into a typed `ApiProblem` and, on any `401`, dispatches `sessionEnded()`.
- **Enhancements.** `src/api/bankApi.ts` calls `enhanceEndpoints` on the generated API to add cache tags (`Accounts`, `History`, `Transfers`, `Me`), the `Idempotency-Key` header on `executeTransfer`, and the infinite-query settings for history.
- **Store.** `src/store/store.ts` exports `makeStore()`, which combines `bankApi.reducer`, `sessionSlice` and `transferDraftSlice`, with RTK Query's middleware and a listener middleware.
- **Provider.** `StoreProvider`, a client component in the root layout, creates one store per browser tab with `useRef`, as the Redux guide for the Next.js App Router recommends. A module-level store would leak between requests on the server.
- **Typed hooks.** Components use `useAppSelector` and `useAppDispatch` from `src/store/hooks.ts` and the generated RTK Query hooks. They never call `fetch` directly.

```
// src/store/store.ts
export const makeStore = () =>
  configureStore({
    reducer: {
      [bankApi.reducerPath]: bankApi.reducer,
      session: sessionSlice.reducer,
      transferDraft: transferDraftSlice.reducer,
    },
    middleware: (getDefault) =>
      getDefault().prepend(listener.middleware).concat(bankApi.middleware),
  });
export type AppStore = ReturnType<typeof makeStore>;
export type RootState = ReturnType<AppStore["getState"]>;
export type AppDispatch = AppStore["dispatch"];
```

Screens and routes

Seven routes cover the typical visit: sign in, review balances, scan history, transfer, check the result.

Route	Screen	Req.	API calls
`/login`	Sign in	01	`POST /auth/login`
`/accounts`	Accounts overview: each account's balance and the total	02	`GET /accounts`
`/accounts/[accountId]`	Account header plus transaction history with filters, search, paging and running balance	02, 03	`GET /accounts/{id}`, `GET /accounts/{id}/transactions`

Route	Screen	Req.	API calls
`/transfer`	Transfer form: source, destination (own account or payee), amount, note	04	`GET /accounts`, `GET /payees`, `POST /transfers/quotes`
`/transfer/confirm`	Confirmation: review the quote, with an expiry countdown	04	`POST /transfers` with `Idempotency-Key`
`/transfers/[transferId]`	Result: reference, amount, balances after; safe to refresh	04	`GET /transfers/{id}`
`/transfers`	Recent transfers	04	`GET /transfers`

Every route except `/login` sits in a `(bank)` route group whose layout holds the header (customer name, navigation, Sign out) and the session check. `/` redirects to `/accounts`.

Sign-in and protected routes

The API's session is the only source of truth: the app knows who is signed in by asking `GET /auth/me`, never by keeping its own token or flag.

- **Sign in.** The form (React Hook Form and Zod, both fields required) calls `useLoginMutation`. On `200`, a `signedIn(customer)` action fills `sessionSlice`, `upsertQueryData` seeds the `me` query, and the app navigates to `next` or `/accounts`. `401 invalid_credentials` shows one generic message under the form. `423 account_locked` and `429 rate_limited` show their own messages; the latter includes the `Retry-After` wait.
- **Session slice.** `sessionSlice` holds `{ status: "unknown" | "signedIn" | "signedOut", customer }`. Only RTK Query results and the base query's 401 handling write it, so it always mirrors the API.

- **Edge pre-check.** `proxy.ts` (the Next.js 16 name for middleware) runs before protected pages render. It only checks that the `obk_session` cookie exists, with no network call, and redirects to `/login?next=<current path>` when it is missing. It cannot tell whether the session is still alive, so the API stays the real gate.

```
// proxy.ts
import { NextResponse, type NextRequest } from "next/server";

export function proxy(req: NextRequest) {
  if (!req.cookies.has("obk_session")) {
    const url = new URL("/login", req.url);
    url.searchParams.set("next", req.nextUrl.pathname +
req.nextUrl.search);
    return NextResponse.redirect(url);
  }
  return NextResponse.next();
}

export const config = {
  matcher: ["/accounts/:path*", "/transfer/:path*", "/transfers/:path*"],
};
```

- **No redirect loops.** `/login` is left out of the matcher, and the app never redirects away from `/login` just because a cookie exists, since a stale cookie would bounce between the two.
- **Session guard.** The `(bank)` layout renders a `SessionGate` client component. It runs `useGetCurrentCustomerQuery()` (`GET /auth/me` , with `refetchOnFocus` turned on through `setupListeners`) to load the customer into `sessionSlice` . While the status is `unknown` it shows a skeleton. On `signedOut` it replaces the route with `/login?next=<current path>` , as a backstop for a cookie that is present but no longer valid.
- **Expiry mid-visit.** When any request returns `401` , `bankBaseQuery` dispatches `sessionEnded()` . A listener resets the API cache (`bankApi.util.resetApiState()`) and the transfer draft, and calls `POST /api/v1/auth/logout` to clear the stale `HttpOnly` cookie, which JavaScript cannot remove itself. `SessionGate` then redirects, and the next page load fails the edge pre-check cleanly. An idle timeout (15 minutes in the API) therefore lands on the sign-in page rather than on half-loaded screens.
- **Sign out.** The button calls `useLogoutMutation` (always `204`), then dispatches `sessionEnded()` for the same cleanup and replaces the route with `/login` .

- **Data isolation.** Handled by the API, which returns `404` for another customer's ids. The UI shows a not-found state and never offers ids it did not get from the API.

Accounts and transaction history

The overview is one call, and the history screen is one infinite query whose key includes the filters. A filter change therefore starts a fresh cursor chain, as the API requires.

Accounts overview (`/accounts`)

- One card per account from `GET /accounts` : name, masked number, type badge, and balance, with checking first as the API orders them. The total balance sits above the cards.
- Each card links to `/accounts/[accountId]` . A "Move money" button opens `/transfer` with that account preselected as the source (`?from=<id>`).
- Balances show an "as of" time from `asOf` . The query refetches on window focus and whenever a transfer invalidates the Accounts cache tag.

Transaction history (`/accounts/[accountId]`)

Control	Behavior	API parameter
Date range	Presets (Last 30 days, Last 90 days, All) plus a two-month calendar; a start after the end is blocked in the picker	`from` , `to` (inclusive dates)
Type	Segmented control: All, Debits, Credits	`type`
Search	Text box, debounced 300 ms; sent only at 2 or more characters	`q`
Paging	25 per page through an RTK Query infinite-query endpoint (`getNextPageParam` reads `page.nextCursor`); a "Load more" button, which also loads when it scrolls into view	`pageSize` , `cursor` from `page.nextCursor`

- Filters live in the URL through nuqs (`/accounts/…?from=2026-09-01&type=debit&q=coffee`), so refresh, back and shared links keep them. "Clear filters" resets to newest first, unfiltered.
- Above the list: "312 results" from `totalCount` , and the active filters as removable chips.
- **Columns:** Date (local date and time), Description (with counterparty and note on a second line), Amount, and Balance after. On a phone each entry becomes a two-line row with the same fields.
- **Debit or credit** is shown as text ("Debit" or "Credit") and as a sign on the amount (−200.00, +1,250.00), with color as a third cue only. That meets the requirement without relying on color.
- **Running balance.** The column is labelled "Balance after" with a tooltip: "Your balance right after this transaction." It comes from `runningBalance` , which the API stores per entry, so it stays correct under any filter.
- Entries that came from a transfer link to `/transfers/[transferId]` .
- **Empty states:** "No transactions yet" for an account with no history (chloe's savings) and "No transactions match these filters" with a Clear button.

Transfer flow

Three screens map onto the API's two calls. The form creates a quote, the confirmation screen executes it with one `Idempotency-Key` that survives retries and refreshes, and the result screen reads the finished transfer by id.

1. Form (`/transfer`)

- **From:** the customer's accounts with their balances. **To:** a grouped select with "My accounts" (the source is excluded) and "Payees". **Amount:** a decimal input (`inputMode="decimal"`). **Note:** optional, with a 140-character counter.
- A Zod schema mirrors the API rules for instant feedback: amount above 0, at most 2 decimals, at most the source balance, source different from destination, note of 140 characters or fewer with no control characters. The API stays the authority.
- "Review transfer" calls `useCreateTransferQuoteMutation` . On `201` the app dispatches `quoteReceived({ quote, form })` . The `transferDraftSlice` reducer generates the `Idempotency-Key` with `crypto.randomUUID()` , and the app goes to `/transfer/confirm` .

- **Draft persistence.** A listener middleware writes `transferDraft` to `sessionStorage` (`obk.transfer`) after every change. `StoreProvider` reads it back as preloaded state, so the draft and its key survive a refresh but not a closed tab.
- A `422` from the quote call is mapped onto fields from `errors[].field`, so every failing rule shows next to its input at once.

2. Confirmation (`/transfer/confirm`)

- The page reads the draft with `useAppSelector(selectTransferDraft)`. It shows from, to, amount, fee (0.00), note, the source's balance after, and "This quote expires in 4:12". Buttons: **Confirm transfer**, **Edit** (back to the prefilled form) and **Cancel** (dispatches `draftCleared()`).
- **Confirm** calls `useExecuteTransferMutation` with `{ quoteId, idempotencyKey }`. The enhanced endpoint sends the key as the `Idempotency-Key` header.
- **Double click.** The button is disabled while `isLoading`, and the click handler returns early if a request is already pending.
- **Retry.** The endpoint's base query is wrapped in RTK Query's `retry` (3 attempts, network errors and timeouts only), and every attempt reuses the same key and body. `409 request_in_progress` waits the `Retry-After` seconds, then retries with the same key.
- **Refresh.** The draft, key included, is restored from `sessionStorage` as preloaded state, so a refresh followed by Confirm sends the same key and the API replays the first result instead of moving money again. With no draft, the page redirects to `/transfer`.
- **Expiry.** At 0:00 the Confirm button turns into "Get a new quote", which re-quotes the same values. The new quote gets a new key.

API answer	What the app does
`201` (fresh or `Idempotent-Replayed`)	Dispatch `draftCleared()`. The mutation's `invalidatesTags` refreshes `Accounts`, both accounts' `History` and `Transfers`. Then `router.replace` to `/transfers/{id}`.
`409 quote_already_used`	Dispatch `draftCleared()`, then `router.replace` to `/transfers/{transferId}` from the problem body

API answer	What the app does
`422 quote_expired`	Back to the prefilled form: "That quote expired. Review and confirm again."
`422` rule codes (for example `insufficient_funds`)	Back to the form with the errors on their fields
`422 idempotency_key_reused`	Treated as a bug: dispatch `draftCleared()` and restart the transfer

3. Result (`/transfers/[transferId]`)

- `GET /transfers/{id}` drives a clear success panel: "$200.00 sent to High-Yield Savings", the reference (`TRF-20261021-000123`), the date, the note, and balances after for the source and, for own-account transfers, the destination.
- Links: **View account** (the source's history, where the new debit is at the top) and **Make another transfer**.
- The page only reads data, so refresh and Back are harmless. `router.replace` keeps Back from returning to the confirmation screen.

Errors, formatting and accessibility

The API's Problem Details `code` drives every error message, so the copy is written once in the app and never parsed from English text.

- **One error type.** `bankBaseQuery` turns `application/problem+json` into `ApiProblem { status, code, detail, errors[], transferId?, retryAfter? }`, which RTK Query hooks return as `error`. `src/lib/problemMessages.ts` maps each code from the API's error table to customer-facing copy. Unknown codes fall back to `detail`.
- **Where errors show.** Field errors go on their inputs. Form-level errors appear in an alert at the top of the form. `404` renders the route's `not-found.tsx`. `500` and network failures show a retry panel plus the `traceId` for support. Toasts are only for successes and background refresh failures; a listener on rejected queries raises the latter.
- **Money.** Amounts arrive as JSON numbers with 2 decimals and are shown with `Intl.NumberFormat("en-US", { style: "currency", currency })`. Amounts the customer types are validated as strings (`^\d+(\.\d{1,2})?$`), then sent as numbers. The app does no arithmetic on balances: "balance after" values always come from the API.

- **Dates.** Timestamps are shown in `America/New_York` (the API's bank time zone, via @date-fns/tz), so dates match the API's date filters. Date filters are sent as `YYYY-MM-DD`.
- **Accessibility.** Radix-based components give keyboard and screen-reader support. Debit or credit is never shown by color alone. Inputs have visible labels, and errors use `aria-describedby`. The confirmation screen moves focus to its heading, and the result panel is announced with `role="status"`. Playwright runs axe on every route, with zero serious violations as the bar.
- **Responsive.** One column on phones and a two-column overview from the `md` breakpoint; the history table turns into stacked rows below `sm`.

Project structure

One Next.js project, `OnlineBankingWeb`, next to the API folder in `OnlineBankingApp`, with routes in `app/` and everything else under `src/`.

```
OnlineBankingWeb/
  app/
    layout.tsx                # html shell, fonts, <StoreProvider>,
<Toaster>
    page.tsx                  # redirect to /accounts
    login/page.tsx
    (bank)/
      layout.tsx              # <SessionGate>, header, nav, sign out
      accounts/page.tsx
      accounts/[accountId]/page.tsx
      transfer/page.tsx
      transfer/confirm/page.tsx
      transfers/page.tsx
      transfers/[transferId]/page.tsx
      not-found.tsx
  src/
    store/
      store.ts                # makeStore(), RootState, AppDispatch
      StoreProvider.tsx       # one store per tab (useRef), preloads the
transfer draft
      hooks.ts                # useAppDispatch, useAppSelector
      listeners.ts            # draft persistence, sessionEnded cleanup
(cache, draft, logout), error toasts
    api/
```

```
      baseApi.ts                # createApi + bankBaseQuery (problem parsing,
401, retry)
      bankApi.generated.ts      # generated by @rtk-query/codegen-openapi
(committed)
      bankApi.ts                # enhanceEndpoints: tags, Idempotency-Key
header, history infinite query
    features/
      auth/                     # sessionSlice, LoginForm, SessionGate
      accounts/                 # AccountCard, AccountsSummary
      history/                  # HistoryFilters, HistoryList, HistoryRow,
useHistoryFilters (nuqs)
      transfers/                # transferDraftSlice, transferSchema (Zod),
TransferForm, ConfirmPanel, ResultPanel
    components/ui/              # shadcn/ui components
    lib/                        # money, dates, problemMessages, env
  tests/
    e2e/                        # Playwright specs against the real API
    mocks/                      # MSW handlers
    renderWithStore.tsx         # test helper: real store + providers
  openapi-config.ts             # @rtk-query/codegen-openapi settings
  next.config.ts                # rewrites
  proxy.ts                      # edge pre-check: no obk_session cookie ->
/login
  .env.development.local        # API_ORIGIN, NODE_EXTRA_CA_CERTS
```

Scripts: `dev` (`next dev --experimental-https`), `build` (`next build`), `start`, `lint`, `format`, `typecheck` (`tsc --noEmit`), `test` (Vitest), `test:e2e` (Playwright), and `api:codegen`.

Testing and open questions

Component tests run against MSW mocks typed from the API schema. A Playwright suite then walks the real visit against the running API and its seeded data.

Layer	Tool	Proves
Unit	Vitest	Zod transfer schema matches every API rule; `sessionSlice` and `transferDraftSlice` reducers; money and date formatting; problem-code mapping

Layer	Tool	Proves
Component	Vitest + Testing Library + MSW, rendered with a real store (`renderWithStore`)	Filters update the URL and the query; field errors land on the right inputs; Confirm disables while pending and reuses the key on retry; a 401 clears the cache
End to end	Playwright vs. real API, on the production build	Sign in as alice, see both balances, filter history, transfer 200.00 to savings, balances and both histories update, refresh on confirm does not double-post, sign out blocks `/accounts`
Accessibility	@axe-core/playwright	No serious violations on any route

The E2E suite calls `POST /api/v1/demo/reset` before each run when the API runs in the Demo environment, so balances start from the seeded values.

Open questions

- **Login rate limit behind the proxy.** The API limits sign-ins to 10 per minute per client IP, and through the proxy every browser shares the Next.js server's IP. That's fine for a local demo; a shared deployment needs forwarded-headers support in the API. **Recommended answer (production).** Use layers rather than one number, because a single per-IP limit is either too tight for shared IPs or too loose for attackers.
  - **Per real client IP:** 20 attempts per minute, sliding window, then `429` with `Retry-After`. That's looser than the demo's 10, because offices, universities and mobile carriers put many customers behind one IP, but it still stops a single machine from brute-forcing.
  - **Per username:** 5 failures, then a 15-minute lockout. The API already does this (`MaxFailedLogins`, `LockoutDuration`), and it's the main defense for any one account. To keep an attacker from locking out real customers on purpose, a growing delay (1s, 2s, 4s…) or a CAPTCHA after 3 failures can replace the hard lock.
  - **Count failures only.** Successful sign-ins don't count against the IP limit, which keeps busy shared IPs working.

- **Behind the proxy:** the API adds `UseForwardedHeaders` with the Next.js server in `KnownProxies`, and Next.js forwards `X-Forwarded-For`. Otherwise every customer shares one IP and the limit is meaningless.
- **More than one API instance:** the built-in limiter counts per instance, so move the counters to Redis or enforce the IP limit at a gateway or WAF (Azure Front Door, Cloudflare and similar).
- **Out of scope for the prototype:** rate limits won't stop credential stuffing from botnets spread across thousands of IPs. A real bank needs MFA, checks of passwords against breach lists, and bot detection for that.